OS VCS PROJECT
NAME: S N Niharika
SRN: PES2UG24CS414

Github link:
https://github.com/snniharika/PES2UG24CS414-pes-vcs

PHASE 1:

Screenshot 1A:

```
seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$ nano object.c
seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$ make clean
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$ make test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function ‘object_write’:
object.c:103:47: warning: ‘/tempXXXXXX’ directive output may be truncated writing 11 bytes into a region of size between
 1 and 512 [-Wformat-truncation=]
  103 |     snprintf(temp_path, sizeof(temp_path), "%s/tempXXXXXX", dir_path);
      |                                              ^~~~~~~~~~~
In file included from /usr/include/stdio.h:980,
                 from pes.h:8,
                 from object.c:11:
In function ‘snprintf’,
    inlined from ‘object_write’ at object.c:103:5:
/usr/include/x86_64-linux-gnu/bits/stdio2.h:54:10: note: ‘__builtin___snprintf_chk’ output between 12 and 523 bytes into
 a destination of size 512
   54 |   return __builtin___snprintf_chk (__s, __n, __USE_FORTIFY_LEVEL - 1,
      |          ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
   55 |                                    __glibc_objsize (__s), __fmt,
      |                                    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
   56 |                                    __va_arg_pack ());
      |                                    ~~~~~~~~~~~~~~~~~
gcc -o test_objects test_objects.o object.o -lcrypto
seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$
```

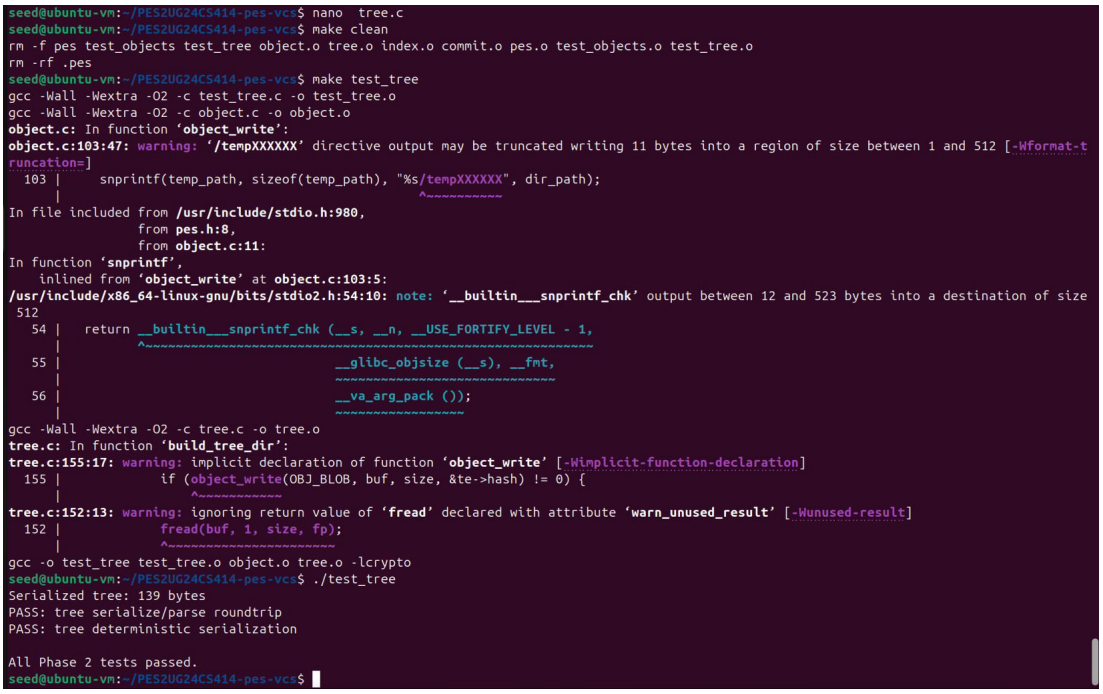
Screenshot 1B:

```
seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$
```

Q1.1
We shard objects into subdirectories so no single folder contains millions of files. Flat directories become slow for file lookup, creation, deletion, and directory listing as entries grow very large. Sharding spreads files across many smaller folders, keeping filesystem metadata operations efficient and scalable.
Q1.2
If we write directly to the final path and the program crashes or power fails midway, the object file may be incomplete or corrupted. Another process could also read a half-written file. Using temp-write then rename ensures only a fully written file becomes visible. POSIX guarantees rename() within the same filesystem is atomic, meaning readers see either the old file or the new file, never a partial state.
Q1.3
Using the birthday paradox, collision probability is approximately $n^2 / (2 \times 2^{bits})$ for n = 1 billion objects. For SHA-1, the chance is about 3.4×10^{-31}. For SHA-256, the chance is about 4.3×10^{-60}, vastly smaller. Git is migrating to SHA-256 because SHA-1 has known practical collision attacks, while SHA-256 provides much stronger long-term security and integrity.

PHASE 2:

Screenshot 2A:



```
seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$ nano  tree.c
seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$ make clean
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes
seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$ make test_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function ‘object_write’:
object.c:103:47: warning: ‘/tempXXXXXX’ directive output may be truncated writing 11 bytes into a region of size between 1 and 512 [-Wformat-t
runcation=]
  103 |     snprintf(temp_path, sizeof(temp_path), "%s/tempXXXXXX", dir_path);
      |                                                ^~~~~~~~~~~
In file included from /usr/include/stdio.h:980,
                 from pes.h:8,
                 from object.c:11:
In function ‘snprintf’,
    inlined from ‘object_write’ at object.c:103:5:
/usr/include/x86_64-linux-gnu/bits/stdio2.h:54:10: note: ‘__builtin___snprintf_chk’ output between 12 and 523 bytes into a destination of size
 512
   54 |   return __builtin___snprintf_chk (__s, __n, __USE_FORTIFY_LEVEL - 1,
      |          ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
   55 |                                    __glibc_objsize (__s), __fmt,
      |                                    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
   56 |                                    __va_arg_pack ());
      |                                    ~~~~~~~~~~~~~~~~~
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
tree.c: In function ‘build_tree_dir’:
tree.c:155:17: warning: implicit declaration of function ‘object_write’ [-Wimplicit-function-declaration]
  155 |             if (object_write(OBJ_BLOB, buf, size, &te->hash) != 0) {
      |                 ^~~~~~~~~~~
tree.c:152:13: warning: ignoring return value of ‘fread’ declared with attribute ‘warn_unused_result’ [-Wunused-result]
  152 |             fread(buf, 1, size, fp);
      |             ^~~~~~~~~~~~~~~~~~~~~~~
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$
```

2b :

```
seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$ xxd .pes/objects/d4/f59a6f74ea3247028d6ddbd8477f482b02b71f166b386a304e9d8ea8840642 | head -20
00000000: 7472 6565 2031 3234 3000 3430 3030 3020  tree 1240.40000
00000010: 2e67 6974 0068 d161 7cbc de28 8463 704e  .git.h.a|..(.cpN
00000020: 90b7 c309 f6fa d1fe 9317 74c4 ceae 3a31  ..........t...:1
00000030: 91b0 4662 4831 3030 3634 3420 2e67 6974  ..FbH100644 .git
00000040: 6967 6e6f 7265 003d d9f8 32ab 4ce5 0c29  ignore.=..2.L..)
00000050: 641e 8fdb 7bc1 c4f5 a471 0008 008c e332  d...{....q.....2
00000060: e258 5767 1d85 ac31 3030 3634 3420 4d61  .XWg...100644 Ma
00000070: 6b65 6669 6c65 0034 3d34 bf69 0fef a64d  kefile.4=4.i...M
00000080: c0d9 5a01 280d f0e5 d489 c8ec ed21 3340  ..Z.(........!3@
00000090: be03 ea5f facc 2b31 3030 3634 3420 5245  ..._..+100644 RE
000000a0: 4144 4d45 2e6d 6400 6045 9622 a6a9 9c40  ADME.md.`E."...@
000000b0: ddfc d668 e7cb d0d7 0a4d 53ae b448 a251  ...h.....MS..H.Q
000000c0: aa98 2ea2 043e 3838 3130 3036 3434 2061  .....>88100644 a
000000d0: 2e74 7874 002c f8d8 3d9e e295 43b3 4a87  .txt.,..=...C.J.
000000e0: 7274 21fd ecb7 e3f3 a183 d337 6390 25de  rt!........7c.%.
000000f0: 576d b9eb b431 3030 3634 3420 636f 6d6d  Wm...100644 comm
00000100: 6974 2e63 00db 441e 44fb cf5e 57c0 2dbd  it.c..D.D..^W.-.
00000110: 2c6c 6672 d91a 3123 4e76 53d5 5dde d677  ,lfr..1#NvS.]..w
00000120: 902b 7bff 3b31 3030 3634 3420 636f 6d6d  .+{.;100644 comm
00000130: 6974 2e68 006b 1a89 bbbd 6a60 4d67 5a2f  it.h.k....j`MgZ/
seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$
```

Q2.1
Tree entries must be sorted so the same directory always produces the same serialized bytes and therefore the same hash. If entries depended on filesystem enumeration order, two developers with identical files could generate different tree objects and different commit hashes. That would break determinism and make identical snapshots look different.
Q2.2
The VCS tracks 100644 vs 100755 so it knows whether a file is executable. This is important for scripts, tools, and binaries that must run directly after checkout. If permission bits were ignored, executable files could be restored as normal files and fail with permission denied errors.
Q2.3
A tree object stores entries that point to files or subtrees. An empty directory has no entries, so there is nothing to serialize and no object reference to represent it. Since the system records content through references, a directory with no contents leaves no trace to store, making empty directories fundamentally untracked.

PHASE 3:

3a:

```
seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$ echo "hello" > file1.txt
seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$ V
V: command not found
seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$ echo "world" > file2.txt
seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$ V
V: command not found
seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$ ./pes add file1.txt file2.txt
seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  object.c
  untracked:  pes.c
  untracked:  test_objects.c
  untracked:  Makefile
  untracked:  index.h
  untracked:  tree.c
  untracked:  test_sequence.sh
  untracked:  .gitignore
  untracked:  pes.h
  untracked:  commit.h
  untracked:  tree.h
  untracked:  index.c
  untracked:  a.txt
  untracked:  commit.c
  untracked:  test_tree.c
  untracked:  README.md
```

3b:

```
seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776753533 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776753541 6 file2.txt
seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$
```

Q3.1
The index stores mtime and size so `pes status` can quickly detect unchanged files without rehashing every file's contents. If both values match, the file is assumed unchanged, which makes status much faster. Worst case: a file is edited but ends up with the same size and timestamp granularity does not change, so the modification is missed until a full hash check is done.

Q3.2
Index entries must be sorted by path so the same staged state always produces the same index file layout. If entries were saved in random order, identical staged files could create different index contents on different systems. That would reduce consistency, make diffs noisy, and break deterministic behavior.

Q3.3
Atomic save is critical because the index is the staging area for the next commit. If writing directly to `.pes/index` crashes halfway, the file may become truncated or partially corrupted, losing staged entries or making it unreadable. Then the repository may have an invalid or incomplete staging state, causing wrong commits or load errors.

PHASE 4:

4a:

```
seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$ ./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes commit -m "first commit"

echo "second line" >> file1.txt
./pes add file1.txt
./pes commit -m "second commit"

echo "new file" > file3.txt
./pes add file3.txt
./pes commit -m "third commit"

./pes log
Initialized empty PES repository in .pes/
Committed: 29a00f6e31b8... first commit
Committed: 029787680de6... second commit
Committed: 48fdfa1fb240... third commit
commit 48fdfa1fb240765dc07a8052554e16548ec698fee843640886d6f622988e8e1e
Author: S N Niharika <PES2UG24CS414>
Date:   1776755056

    third commit

commit 029787680de647159787e2a9e5bc4fd5477000122c168553fc51acc97d99c51a
Author: S N Niharika <PES2UG24CS414>
Date:   1776755056

    second commit

commit 29a00f6e31b8e165a5496a78aa0def9ce775526a524050eba67e16e5363dd6f1
Author: S N Niharika <PES2UG24CS414>
Date:   1776755055

    first commit

seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$
```

4b:

```
seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/00/ef76339ec9037887c5fa8df372e4597a9ed72437f4005a3013a85bf85470e7
.pes/objects/02/9787680de647159787e2a9e5bc4fd5477000122c168553fc51acc97d99c51a
.pes/objects/02/d663c03330e7ec172a16b38b8bb05de3ac3553489f2c7d0065a69413cea149
.pes/objects/04/535a3006c6fe78d8cfadeacc6f5c219c7809be133f4d1780b777630686834a
.pes/objects/05/3e9f8ea2ab3d9adfcdd1825326f27face3381a5e26fff1ffaba9d9d4d5289b
.pes/objects/06/604861a834e4ea1fcdfd8a8637979f22d3304b857d7b2887fdcfc877596448
.pes/objects/0f/d4cc0c81f6e10038a7d1f6f83fa1a0bc2d384247b06ebcd90b37e65e95d039
.pes/objects/10/d488e897a9291aca9f7f55db526cf1c0a4d2a6274de8d6729d8da036cce9fd
.pes/objects/15/5a7fcbf850f594e288661d422c337ac73c108ca94de05bb08c7a2c6b316585
.pes/objects/15/636d39be47a9cca645e76a78fbb2dee64c98536ffd8f2e7824c0e932a78232
.pes/objects/17/237a3a677193a33d768957fb3bd09b9cc7f74048eafe8ddfa18024d87e2c15
.pes/objects/1a/637a22a51adcadb6b62d1af1b76e47e2e212cbb9126a0d52899a8a0a43b358
.pes/objects/1c/489cbd16de43599db722c8105cae1d10a4ac8d125814b909f5f721acbb4df2
.pes/objects/1c/7194cab317116072fa4766ca6da2d1c85a32c6bff2880dee4627c4ab6bf220
.pes/objects/1f/3a95a313f1868a019c63d80f56737b0322c44b1e0201192f9acc66dd863483
.pes/objects/1f/f2273de695f67d7f76daf405ef20d448c5d806eff0d139829795d5a94dd56f
.pes/objects/23/93dd1b5b22510ca4aac4b6ffc654a598fe27475228c9a19cf5cd85e5cccd5e
.pes/objects/27/a69c6dedcc2c164019092ba4f2c05d8a7a418ef0a84d2bdb3922c65cf0abde
.pes/objects/29/a00f6e31b8e165a5496a78aa0def9ce775526a524050eba67e16e5363dd6f1
.pes/objects/2b/e681526475c1d77382e34417538c2163f3b714ab47848f4b69cf3b03cdb032
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/34/1825af008c12574870e8ad654a05eb1fe2fbf058f572ea1345d016a74f9df5
.pes/objects/34/3d34bf690fefa64dc0d95a01280df0e5d489c8eced213340be03ea5ffacc2b
.pes/objects/39/4b1b447d2bc693d82619f7c157f4d969cd7634780eb531550c88676b0a9625
.pes/objects/3b/34f0f39242830cfb603d169be26d0be261b0bf0bfd8bf21d6f19944a52c200
.pes/objects/3d/d9f832ab4ce50c29641e8fdb7bc1c4f5a4710008008ce332e25857671d85ac
.pes/objects/42/359f28504398314d1dfe01f0ef7e0c9ccdfadf2db61c8e6232ed7fd3ff0bfc
.pes/objects/48/fdfa1fb240765dc07a8052554e16548ec698fee843640886d6f622988e8e1e
.pes/objects/49/9eb307eecffbbe1e13c80a077f422d5d4bc3e3cfd31594694ec2d13eff0c27
.pes/objects/4a/2b81551cd12a05224061a6148bd2d0e2cdba83bff0d1c9531edf848c9588f5
.pes/objects/51/274a32ab6987890333d71be6deb0376692b305b2dfd02071633c42ec490584
.pes/objects/52/a0bc45493f47914485db9ddb151e20a278316b7227660e2e845c81b84d5673
.pes/objects/57/651cafa21326dd46b7de755490b53b1d46f664cf17038846b17929fe042b6d
.pes/objects/57/8456144fef1c8e61adb167a7496c132f9089254f340c4770c1c67a1e2444e7
.pes/objects/59/05a5051519324de161183ea26eee3b411ee6b88de3f48242332c04cedff1ee
.pes/objects/59/4c12541ff31bf41dc2a4713aeeb4f25c8e36563388fb869c50f9400096026d
.pes/objects/59/b136237920e9fa871ff25fa3e3a9872a3baaef1c350927beab6d061331005b
```

4c:

```
seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$ cat .pes/HEAD
cat .pes/refs/heads/main
ref: refs/heads/main
48fdfa1fb240765dc07a8052554e16548ec698fee843640886d6f622988e8e1e
seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$
```

Q4.1
The commit object may already exist in the object store, but history only follows commits reachable from branch refs. When `.pes/refs/heads/main` is updated to the new hash, the branch now points to that commit and future logs can reach it. That file update is the moment the commit becomes visible part of history.
Q4.2
The new commit object exists on disk, but the branch still points to the old commit, so history appears unchanged. The commit is usually recoverable if its hash is known or found before garbage collection removes unreachable objects. This is safe because no existing history is corrupted; only the new commit was not published.
Q4.3
If overwrite-in-place crashes after writing only 32 of 64 hex characters, the ref file becomes invalid or truncated. Then HEAD cannot resolve the branch correctly, and log/commit operations may fail or point nowhere. Using temp-write then rename prevents readers from ever seeing a partially written ref.

Full integration test:

```
seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 8cb545a1b41a... Initial commit

Log after first commit:
commit 8cb545a1b41a8e349eee14fb16233cde52b42a01626603c7661fce78fd3e3730
Author: S N Niharika <PES2UG24CS414>
Date:   1776756159

    Initial commit


--- Second Commit ---
Committed: a44c08f33eab... Update file.txt

--- Third Commit ---
Committed: 716f4d599aa0... Add farewell
```

```
--- Full History ---
commit 716f4d599aa036b022fc72d5fd7d6794c98e2fc47d40ef4f80a122f901918476
Author: S N Niharika <PES2UG24CS414>
Date:   1776756160

    Add farewell

commit a44c08f33eabe428d36721e7e3c7418df1ec6e386750ccf1570ee3465cefe080
Author: S N Niharika <PES2UG24CS414>
Date:   1776756160

    Update file.txt

commit 8cb545a1b41a8e349eee14fb16233cde52b42a01626603c7661fce78fd3e3730
Author: S N Niharika <PES2UG24CS414>
Date:   1776756159

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
716f4d599aa036b022fc72d5fd7d6794c98e2fc47d40ef4f80a122f901918476

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/71/6f4d599aa036b022fc72d5fd7d6794c98e2fc47d40ef4f80a122f901918476
.pes/objects/8c/b545a1b41a8e349eee14fb16233cde52b42a01626603c7661fce78fd3e3730
.pes/objects/a4/4c08f33eabe428d36721e7e3c7418df1ec6e386750ccf1570ee3465cefe080
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
seed@ubuntu-vm:~/PES2UG24CS414-pes-vcs$
```

PHASE 5:

Q5.1
pes checkout <branch> would update HEAD to point to .pes/refs/heads/<branch>, then read that branch’s latest commit and restore its tree into the working directory. Files not in the target branch must be deleted, changed files overwritten, and new files created. The complexity comes from safely replacing many files while preserving user data and handling conflicts, nested directories, and partial failures.

Q5.2
First compare the current working files with the index entries (hash/mtime/size) to detect modified tracked files. Then compare the target branch tree with the current branch tree to find files that would change during checkout. If a file is both locally modified and would be replaced by checkout, refuse the operation. This prevents losing uncommitted work.

Q5.3
In detached HEAD, new commits are created normally, but no branch name moves forward to reference them. If the user switches away, those commits can become unreachable later. Recovery is done by creating a branch/tag pointing to that commit hash before garbage collection removes it.

PHASE 6:

Q6.1 Start from all branch heads in .pes/refs/heads/ and traverse every reachable commit, tree, and blob using DFS/BFS. Store visited hashes in a hash set for fast lookup. After traversal, scan .pes/objects; anything not in the reachable set can be deleted. With 100,000 commits and 50 branches, you visit roughly all unique reachable objects once, not 50× duplicates.

Q6.2 If GC runs during commit creation, it may delete a newly written tree/blob before the new commit updates HEAD to reference it. Then the commit points to missing objects and the repo becomes corrupted. Real Git avoids this using locks, temporary protection of new objects, atomic ref updates, and conservative pruning (usually only old unreachable objects).